

Control Statements

control statements allow specific parts of the program to be executed under certain conditions - control the flow of program

if statement: if the test condition in () is true then the **codeblock {}** is executed, eg.

```
int i = 1, k = 2;  
if( k > i ) { codeblock  
    cout << "\nk is greater than i";  
}
```

the test condition () is evaluated with the current value of the variables at that line in the program -- results can change depending on the line in the program

{} isn't required for codeblocks with only one program line, eg
if (i > 0) cout << "\ni is greater than zero";

Test Conditions

The comparison operators below can be used in test conditions:

greater than, less than: $>$, $<$

greater than or equal, less than or equal: $\geq$, $\leq$

equal, not equal: $==$, $!=$

```
if( i  $\geq$  k ) cout << "\ni is greater than or equal to k";
```

```
if( i == k ) cout << "\ni is equal to k";
```

equality conditions should normally not be used with float / double variables -- round off error makes execution unlikely, eg

```
// if statement is executed only if x = 0.00000000000000000000e0
```

```
if( x == 0.0 ) cout << "\nx is equal to zero";
```

thus the if statement might fail even if x is very small and equal to zero for engineering purposes

in order to test for approximate equality for floats / doubles the following approach should be used

```
double eps = 1.0e-9; // a small positive constant epsilon
if( abs(x) < eps ) cout << "\nx is approximately equal to zero";
```

the required value of eps depends on the engineering application

to check if x is equal to 5.5 for example, you can write

```
if( abs(x-5.5) < eps ) cout << "\nx is approximately equal to 5.5";
```

The `<=` and `>=` operators should also normally be avoided for float / double variables -- the approach above can also be modified for this case

Logical Operators

comparison operators can be combined using logical operators for more complex test conditions in if statements, eg

```
if( (i > k) && (k <= 3) ) {  
    cout << "(i is greater than k) AND (k is less than or equal to 3)";  
}
```

The following logical operators are available in C++:

AND: &&

- both conditions have to be true for the result to be true

OR: ||

- only one of the conditions have to true for the result to be true

NOT: !

- the condition has to be false for the result to be true, eg

```
if( !(i > k) ) cout << "\ni > k is false";
```

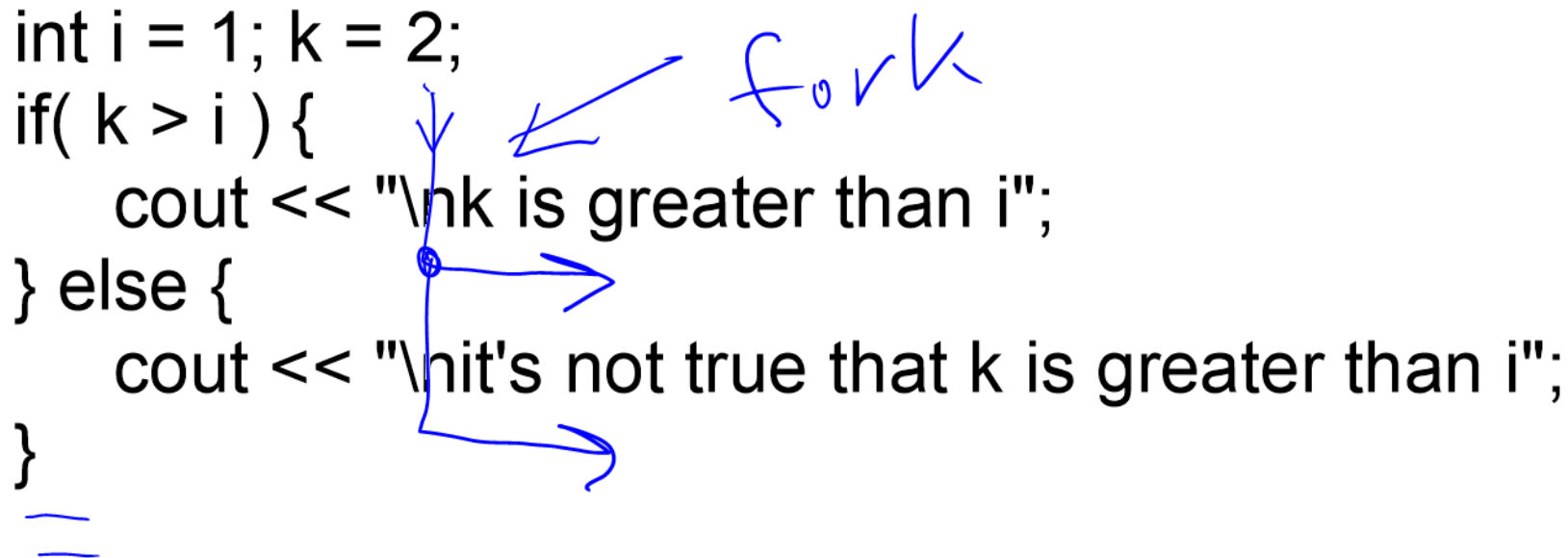
multiple ANDs and ORs are possible in an expression, eg

```
if( (i > k) && (k <= 3) && (i < 10) ) {  
    cout << "\n(i > k), (k <= 3), and (i < 10) are all true";  
}
```

if - else statement

if the test condition in () is true then the 1st codeblock is executed, otherwise the 2nd codeblock is executed, eg.

```
int i = 1; k = 2;  
if( k > i ) {  
    cout << "\nk is greater than i";  
} else {  
    cout << "\nit's not true that k is greater than i";  
}  
      
```



the else statement is like a default option that occurs when the if statement is not true

an if-else statement is like a "fork in the road" since the program must follow one of the two code paths / blocks

in practice if and if-else statements are good for decision making, eg

```
if ( robot is not at destination ) {  
    // move robot forward  
} else { // otherwise, robot is at destination  
    // stop robot  
}
```

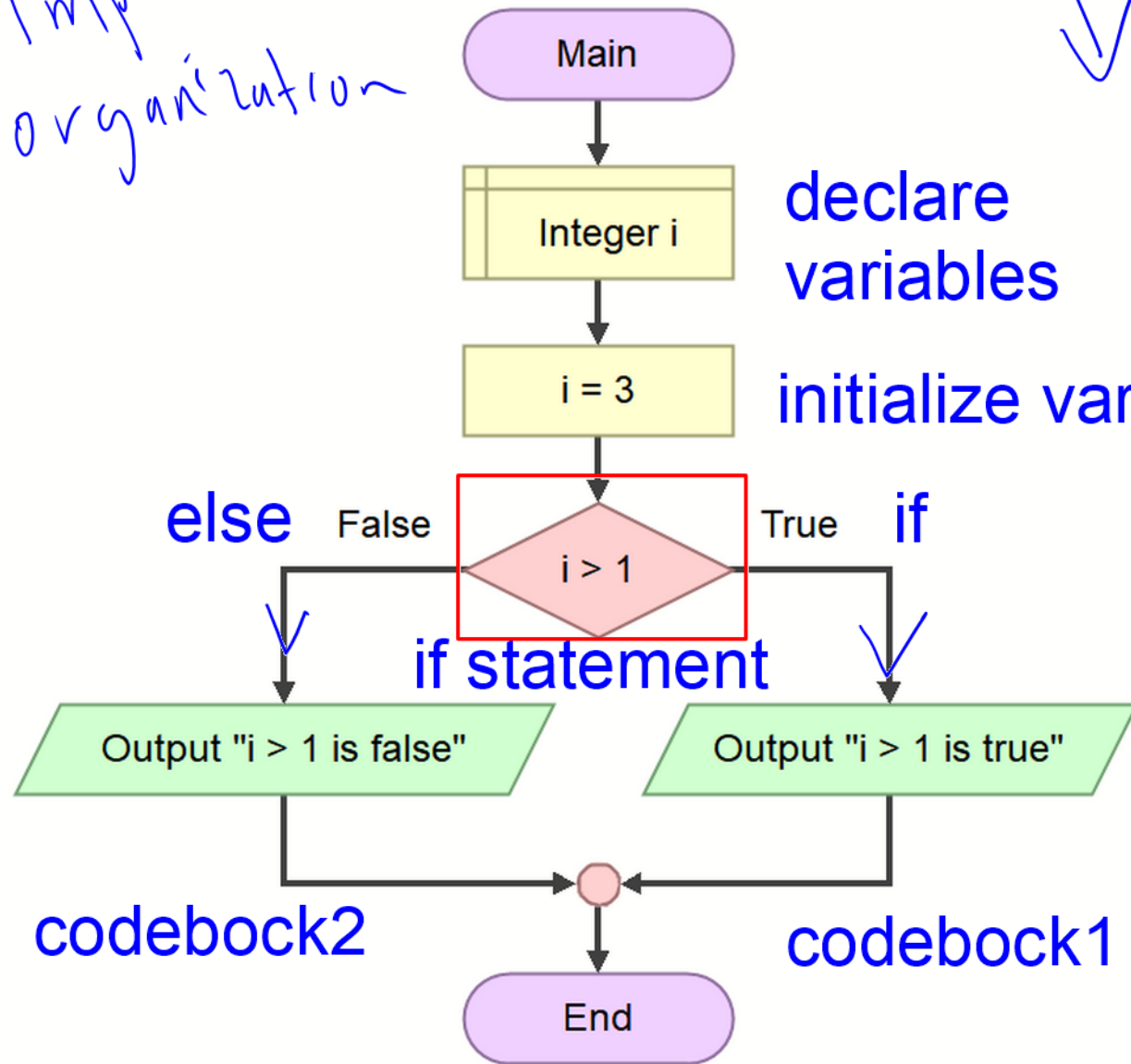
$d > 0$

$d = 0$

Flow Chart Representation

improve organization

presentations



A universal programming language

C++ Java Python

made with Flowgorithm software package (it's free !)